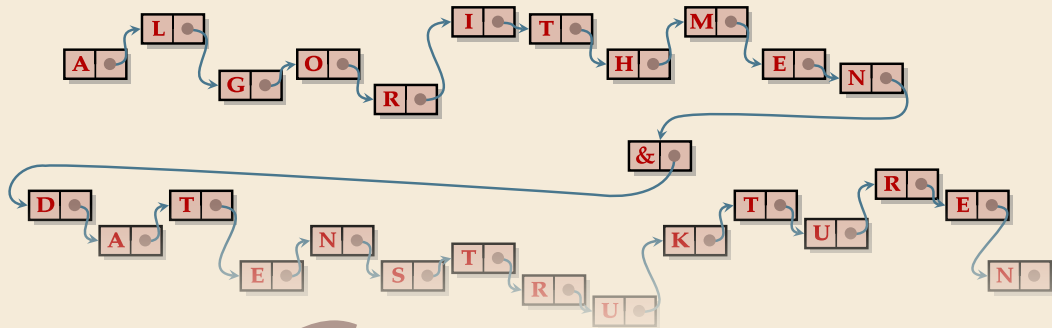




6

Algorithm Science

Algorithmen & Datenstrukturen · Sommersemester 2026

Prof. Dr. Sebastian Wild

6 Algorithm Science

- 6.1 Algorithm Science
- 6.2 Laufzeitexperimente
- 6.3 Mathematische Analyse von Algorithmen
- 6.4 Typische Wachstumsraten
- 6.5 Beispiel: Suchen in einem Array
- 6.6 Speicherbedarf

6.1 Algorithm Science

Modern take on studying algorithms

algorithm science

*The application of the **scientific method** for the design and analysis of algorithms – developing **mathematical models** for characteristics of algorithms and their inputs, formulating hypotheses relevant to the study of performance on specific classes of computers and systems, running **experiments to validate the hypotheses**, and iterating as appropriate with the goal of improving performance.*

<https://sedgewick.io/ideas/#algorithm-science>

The Scientific Method

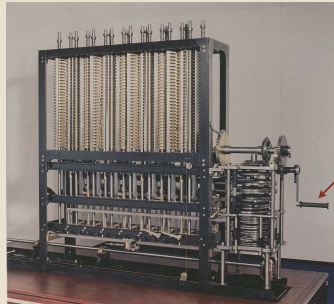
Hypothese — Vorhersage — Experimentelle Überprüfung



► Feynman on Scientific Method.
<https://youtu.be/EYPapE-3FRw>

Running time

“ As soon as an Analytic Engine exists, it will necessarily guide the future course of the science. Whenever any result is sought by its aid, the question will arise—By what course of calculation can these results be arrived at by the machine in the shortest time? ” — Charles Babbage (1864)



Analytic Engine

Cast of characters



Programmer needs to develop a working solution.



Student might play any or all of these roles someday.



Client wants to solve problem efficiently.



Theoretician wants to understand.

Reasons to analyze algorithms

Predict performance.

Compare algorithms.

Provide guarantees.

Understand theoretical basis.

this course (COS 226)

theory of algorithms (COS 423)

Primary practical reason: avoid performance bugs.



client gets poor performance because programmer
did not understand performance characteristics



The challenge

Q. Will my program be able to solve a large practical input?



Insight. [Knuth 1970s] Use **scientific method** to understand performance.

Scientific method applied to analysis of algorithms

A framework for predicting performance and comparing algorithms.

Scientific method.

- **Observe** some feature of the natural world.
- **Hypothesize** a model that is consistent with the observations.
- **Predict** events using the hypothesis.
- **Verify** the predictions by making further observations.
- **Validate** by repeating until the hypothesis and observations agree.

Principles.

- Experiments must be **reproducible**.
- Hypotheses must be **falsifiable**.

Feature of the natural world. Computer itself.



6.2 Laufzeitexperimente

Ein (erstes) Wiedersehen: 3-SUM

3-SUM-Problem

- ▶ **Gegeben:** $A[0..n)$ with $A[i] \in \mathbb{Z}$ for $0 \leq i < n$.
- ▶ **Ziel:** Anzahl c von Tripeln aus A mit Summe 0

Parameter:

- ▶ Eingabegröße: # Zahlen n

Ein (erstes) Wiedersehen: 3-SUM

3-SUM-Problem

- ▶ **Gegeben:** $A[0..n)$ with $A[i] \in \mathbb{Z}$ for $0 \leq i < n$.
- ▶ **Ziel:** Anzahl c von Tripeln aus A mit Summe 0

Parameter:

- ▶ Eingabegröße: # Zahlen n

Mit der einfachsten Lösung hatten wir auch schon vorgearbeitet:

1. 💡 **Algorithmische Idee**
Alle Tripel durchprobieren

2. </> **Pseudocode**
(3 geschachtelte for-Schleifen)
siehe nächste Folie

3. ☉ **Korrektheitsbeweis**
direkt nach Definition von c

4. 🏔 **Analyse**
Ausführungen von Zeilen 5–6

$$\frac{(n-2)(n-1)n}{6} \sim \frac{1}{6}n^3 = \Theta(n^3)$$

Brute-Force 3-SUM Code

Pseudocode

```
1 c = 0
2 for i = 0, ..., n-3           (i, j, k)
3   for j = i+1, ..., n-2
4     for k = j+1, ..., n-1
5       t = A[i] + A[j] + A[k]
6       if t == 0 then c := c + 1
7     end for
8   end for
9 end for
10 return c
```

$$10^{-9}$$

$$\frac{1}{6} \cdot (1000)^3 = \frac{1}{6} \cdot 10^9$$

Java-Implementierung

```
1 import edu.princeton.cs.algs4.*;
2
3 public class ThreeSum {
4     public static int count(int[] a) {
5         int n = a.length;
6         int count = 0;
7         for (int i = 0; i < n-2; i++)
8             for (int j = i+1; j < n-1; j++)
9                 for (int k = j+1; k < n; k++)
10                    if (a[i] + a[j] + a[k] == 0)
11                        count++;
12         return count;
13     }
14
15     public static void main(String[] args) {
16         In in = new In(args[0]);
17         int[] a = in.readAllInts();
18         StdOut.println(count(a));
19     }
20 }
```

Clicker Question

Wie lange dauert es, mit ThreeSum für $n = 1\,000$ Zahlen die Tripel zu zählen, die sich zu 0 summieren?



A 1.3 ns

B 1.3 μ s

C 13 μ s

D 13 ms

E 130 ms

F 1.3 s

G 1.3 min

H 13 min

I 130 min

J 13 h

K 1.3 Tage

L 1.3 Jahre



→ sli.do/cs210

Clicker Question

Wie lange dauert es, mit ThreeSum für $n = 1\,000$ Zahlen die Tripel zu zählen, die sich zu 0 summieren?



A ~~1.3 ns~~

B ~~1.3 μ s~~

C ~~13 μ s~~

D ~~13 ms~~

E 130 ms ✓

F ~~1.3 s~~

G ~~1.3 min~~

H ~~13 min~~

I ~~130 min~~

J ~~13 h~~

K ~~1.3 Tage~~

L ~~1.3 Jahre~~



→ sli.do/cs210

A. Manual.



tick tick tick

tick tick tick tick tick tick tick tick

A pie chart divided into three equal sectors. Two sectors are colored red, and one sector is white. This represents the fraction $\frac{2}{3}$.

[illegible]

13

Measuring the running time

Q. How to time a program?

A. Automatic.

```
public class Stopwatch (part of stdlib.jar)
```

```
    Stopwatch() create a new stopwatch
```

```
    double elapsedTime() time since creation (in seconds)
```

```
public static void main(String[] args)
{
    In in = new In(args[0]);
    int[] a = in.readAllInts();
    Stopwatch stopwatch = new Stopwatch();
    StdOut.println(ThreeSum.count(a));
    double time = stopwatch.elapsedTime();
    StdOut.println("elapsed time " + time);
}
```

Empirical analysis

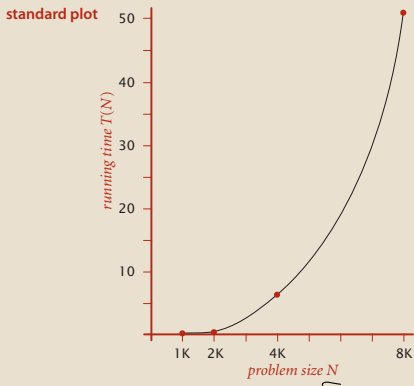
Run the program for various input sizes and measure running time.

N	time (seconds) †
250	0.0
500	0.0
1,000	0.1
2,000	0.8
4,000	6.4
8,000	51.1
16,000	?

$$\frac{5}{6} n^3 \cdot 10^{-9}$$

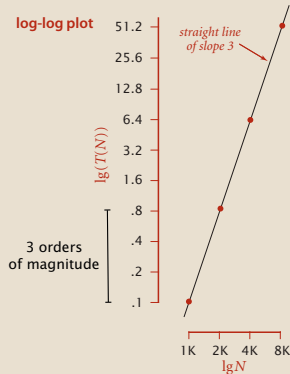
Data analysis

Standard plot. Plot running time $T(N)$ vs. input size N .



Data analysis

Log-log plot. Plot running time $T(N)$ vs. input size N using **log-log scale**.



$$\lg(T(N)) = b \lg N + c$$

$$b = 2.999$$

$$c = -33.2103$$

$$T(N) = a N^b, \text{ where } a = 2^c$$

Regression. Fit straight line through data points: $a N^b$.

Hypothesis. The running time is about $1.006 \times 10^{-10} \times N^{2.999}$ seconds.

power law
slope

Prediction and validation

Hypothesis. The running time is about $1.006 \times 10^{-10} \times N^{2.999}$ seconds.

"order of growth" of running
time is about N^3 [stay tuned]

Predictions.

- 51.0 seconds for $N = 8,000$.
- 408.1 seconds for $N = 16,000$.

Observations.

N	time (seconds) †
8,000	51.1
8,000	51.0
8,000	51.1
16,000	410.8

validates hypothesis!

Doubling hypothesis

Doubling hypothesis. Quick way to estimate b in a power-law relationship.

Run program, **doubling** the size of the input.

N	time (seconds) †	ratio	lg ratio
250	0.0		–
500	0.0	4.8	2.3
1,000	0.1	6.9	2.8
2,000	0.8	7.7	2.9
4,000	6.4	8.0	3.0
8,000	51.1	8.0	3.0

$$\begin{aligned}\frac{T(2N)}{T(N)} &= \frac{a(2N)^b}{aN^b} \\ &= 2^b\end{aligned}$$

$$\lg(6.4 / 0.8) = 3.0$$

seems to converge to a constant $b \approx 3$

Hypothesis. Running time is about aN^b with $b = \lg \text{ratio}$.

Caveat. Cannot identify logarithmic factors with doubling hypothesis.

$$T(N) \approx a \cdot N^3$$

$$\frac{T(2N)}{T(N)} = \frac{a(2N)^3}{aN^3}$$

$$= \frac{8N^3}{N^3} = 8$$

Doubling hypothesis

Doubling hypothesis. Quick way to estimate b in a power-law relationship.

Q. How to estimate a (assuming we know b) ?

A. Run the program (for a sufficient large value of N) and solve for a .

N	time (seconds) †
8,000	51.1
8,000	51.0
8,000	51.1

$$51.1 = a \times 8000^3$$

$$\Rightarrow a = \underline{0.998 \times 10^{-10}}$$

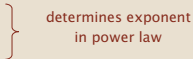
Hypothesis. Running time is about $0.998 \times 10^{-10} \times N^3$ seconds.



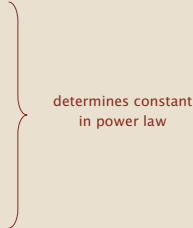
almost identical hypothesis
to one obtained via linear regression

Experimental algorithmics

System independent effects.

- Algorithm.
 - Input data.
- 
- determines exponent
in power law

System dependent effects.

- Hardware: CPU, memory, cache, ...
 - Software: compiler, interpreter, garbage collector, ...
 - System: operating system, network, other apps, ...
- 
- determines constant
in power law

Bad news. Difficult to get precise measurements.

Good news. Much easier and cheaper than other sciences.



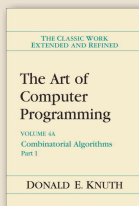
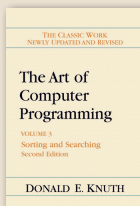
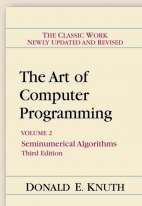
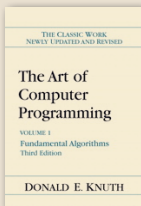
e.g., can run huge number of experiments

6.3 Mathematische Analyse von Algorithmen

Mathematical models for running time

Total running time: sum of cost $\times$ frequency for all operations.

- Need to analyze program to determine set of operations.
- Cost depends on machine, compiler.
- Frequency depends on algorithm, input data.



Donald Knuth
1974 Turing Award

In principle, accurate mathematical models are available.

Cost of basic operations

Challenge. How to estimate constants.

operation	example	nanoseconds †
integer add	$a + b$	2.1
integer multiply	$a * b$	2.4
integer divide	a / b	5.4
floating-point add	$a + b$	4.6
floating-point multiply	$a * b$	4.2
floating-point divide	a / b	13.5
sine	<code>Math.sin(theta)</code>	91.3
arctangent	<code>Math.atan2(y, x)</code>	129.0
...	...	...

† Running OS X on Macbook Pro 2.2GHz with 2GB RAM

Cost of basic operations

Observation. Most primitive operations take constant time.

operation	example	nanoseconds [†]
variable declaration	<code>int a</code>	c_1
assignment statement	<code>a = b</code>	c_2
integer compare	<code>a < b</code>	c_3
array element access	<code>a[i]</code>	c_4
array length	<code>a.length</code>	c_5
1D array allocation	<code>new int[N]</code>	$c_6 N$
2D array allocation	<code>new int[N][N]</code>	$c_7 N^2$

Caveat. Non-primitive operations often take more than constant time.

 novice mistake: abusive string concatenation

Beispiel: Knuthsche Analyse

Wir betrachten folgenden Code für das (wenig spannende) 1-SUM-Problem

```
1 public static int oneSum(int[] a) {  
2     int count = 0, N = a.length;  
3     for (int i = 0; i < N; ++i)  
4         if (a[i] == 0)  
5             count++;  
6     return count;  
7 }
```

oneSum($A[0..n]$)

$N+1$
 N
 $[0..N]$ worst case N
 1

Wie viele Instruktionen
als Funktion in N ?

Beispiel: Knuthsche Analyse

Wir betrachten folgenden Code für das (wenig spannende) 1-SUM-Problem

```
1 public static int oneSum(int[] a) {  
2     int count = 0, N = a.length;  
3     for (int i = 0; i < N; ++i)  
4         if (a[i] == 0)  
5             count++;  
6     return count;  
7 }
```

Wie viele Instruktionen
als Funktion in N ?

Zeile Z	Frequenz $f_Z(N)$
2	1
3	$N + 1$
4	N
5	$[0..N]$
6	1

Beispiel: Knuthsche Analyse

Wir betrachten folgenden Code für das (wenig spannende) 1-SUM-Problem

```
1 public static int oneSum(int[] a) {  
2     int count = 0, N = a.length;  
3     for (int i = 0; i < N; ++i)  
4         if (a[i] == 0)  
5             count++;  
6     return count;  
7 }
```

Wie viele Instruktionen
als Funktion in N ?

Zeile Z	Frequenz $f_Z(N)$
2	1
3	$N + 1$
4	N
5	$[0..N]$
6	1

Analyse aller Operationen

Operation	Frequenz	Kosten
<u>Variablendeklaration</u>	3	c_1
Zuweisung	3	c_2
<u>Vergleich $<$</u>	$N + 1$	c_3
Vergleich $==$	N	c_4
Array-Zugriff	N	c_5
Inkrement	$[N..2N]$	c_6

Beispiel: Knuthsche Analyse

Wir betrachten folgenden Code für das (wenig spannende) 1-SUM-Problem

```
1 public static int oneSum(int[] a) {  
2     int count = 0, N = a.length;  
3     for (int i = 0; i < N; ++i)  
4         if (a[i] == 0)  
5             count++;  
6     return count;  
7 }
```

Wie viele Instruktionen
als Funktion in N ?

Zeile Z	Frequenz $f_Z(N)$
2	1
3	$N + 1$
4	N
5	$[0..N]$
6	1

Analyse aller Operationen

Operation	Frequenz	Kosten
Variablendeklaration	3	c_1
Zuweisung	3	c_2
Vergleich $<$	$N + 1$	c_3
Vergleich $==$	N	c_4
Array-Zugriff	N	c_5
Inkrement	$[N..2N]$	c_6

↪ Gesamtkosten

$$T(N) \leq (c_3 + c_4 + c_5 + 2c_6)\underline{N} + 2(c_1 + c_2) + c_3$$

Beispiel: Knuthsche Analyse

Wir betrachten folgenden Code für das (wenig spannende) 1-SUM-Problem

```
1 public static int oneSum(int[] a) {  
2     int count = 0, N = a.length;  
3     for (int i = 0; i < N; ++i)  
4         if (a[i] == 0)  
5             count++;  
6     return count;  
7 }
```

Wie viele Instruktionen
als Funktion in N ?

Zeile Z	Frequenz $f_Z(N)$
2	1
3	$N + 1$
4	N
5	$[0..N]$
6	1

Analyse aller Operationen

Operation	Frequenz	Kosten
Variablendeklaration	3	c_1
Zuweisung	3	c_2
Vergleich $<$	$N + 1$	c_3
Vergleich $==$	N	c_4
Array-Zugriff	N	c_5
Inkrement	$[N..2N]$	c_6

↪ Gesamtkosten

$$T(N) \leq (c_3 + c_4 + c_5 + 2c_6)N + 2(c_1 + c_2) + c_3$$

alle Instruktionen zählen i.d.R. zu aufwändig

Beispiel: Knuthsche Analyse

Wir betrachten folgenden Code für das (wenig spannende) 1-SUM-Problem

```
1 public static int oneSum(int[] a) {  
2     int count = 0, N = a.length;  
3     for (int i = 0; i < N; ++i)  
4         if (a[i] == 0)  
5             count++;  
6     return count;  
7 }
```

Wie viele Instruktionen
als Funktion in N ?

Zeile Z	Frequenz $f_Z(N)$
2	1
3	$N + 1$
4	N
5	$[0..N]$
6	1

Analyse aller Operationen

Operation	Frequenz	Kosten
Variablendeklaration	3	c_1
Zuweisung	3	c_2
Vergleich $<$	$N + 1$	c_3
Vergleich $==$	N	c_4
Array-Zugriff	N	c_5
Inkrement	$[N..2N]$	c_6

↪ Gesamtkosten

$$T(N) \leq (c_3 + c_4 + c_5 + 2c_6)N + 2(c_1 + c_2) + c_3$$

alle Instruktionen zählen i.d.R. zu aufwändig

↪ Wählen *abstraktes Kostenmaß*: # Arrayzugriffe

Beispiel: Knuthsche Analyse

Wir betrachten folgenden Code für das (wenig spannende) 1-SUM-Problem

```
1 public static int oneSum(int[] a) {  
2     int count = 0, N = a.length;  
3     for (int i = 0; i < N; ++i)  
4         if (a[i] == 0)  
5             count++;  
6     return count;  
7 }
```

Wie viele Instruktionen
als Funktion in N ?

Zeile Z	Frequenz $f_Z(N)$
2	1
3	$N + 1$
4	N
5	$[0..N]$
6	1

Analyse aller Operationen

Operation	Frequenz	Kosten
Variablendeklaration	3	c_1
Zuweisung	3	c_2
Vergleich $<$	$N + 1$	c_3
Vergleich $==$	N	c_4
Array-Zugriff	N	c_5
Inkrement	$[N..2N]$	c_6

↪ Gesamtkosten

$$T(N) \leq (c_3 + c_4 + c_5 + 2c_6)N + 2(c_1 + c_2) + c_3$$

alle Instruktionen zählen i.d.R. zu aufwändig

↪ Wählen abstraktes Kostenmaß: # Arrayzugriffe

↪ Kosten von 1-SUM: $\sim N$

Simplifying the calculations

*“ It is convenient to have a **measure of the amount of work involved in a computing process**, even though it be a very **crude** one. We may count up the number of times that various elementary operations are applied in the whole process and then given them various weights. We might, for instance, count the number of additions, subtractions, multiplications, divisions, recording of numbers, and extractions of figures from tables. In the case of computing with matrices most of the work consists of multiplications and writing down numbers, and **we shall therefore only attempt to count the number of multiplications and recordings.** ” — Alan Turing*

ROUNDING-OFF ERRORS IN MATRIX PROCESSES

By A. M. TURING

(National Physical Laboratory, Teddington, Middlesex)

[Received 4 November 1947]

SUMMARY

A number of methods of solving sets of linear equations and inverting matrices are discussed. The theory of the rounding-off errors involved is investigated for some of the methods. In all cases examined, including the well-known 'Gauss elimination process', it is found that the errors are normally quite moderate: no exponential build-up need occur.



Von Code zu Analyse

Achtung: Im Allgemeinen Ausführungsfrequenz von Codestelle vielleicht unmöglich zu analysieren.

Von Code zu Analyse

Achtung: Im Allgemeinen Ausführungsfrequenz von Codestelle vielleicht unmöglich zu analysieren.

Oft geht es aber, z.B. nach einer der folgenden Regeln

Code	Laufzeit T
<code>x = 0</code> , <code>a[i]</code>	1 (falls Teil des abstrakte Kostenmaßes, sonst 0)
<code>x + y</code> , <code>x * y</code> , <code>x++</code> etc.	

Von Code zu Analyse

Achtung: Im Allgemeinen Ausführungsfrequenz von Codestelle vielleicht unmöglich zu analysieren.

Oft geht es aber, z.B. nach einer der folgenden Regeln

Code	Laufzeit T
$x = 0$, $a[i]$ $x + y$, $x * y$, $x++$ etc.	1 (falls Teil des abstrakte Kostenmaßes, sonst 0)
<pre>1 for (int j = a; j <= b; ++j) { 2 B(j) // weiterer Code 3 }</pre>	$\sum_{j=a}^b T_B(j)$ für $T_B(j)$ Laufzeit des Schleifenrumpfs mit Wert j

Von Code zu Analyse

Achtung: Im Allgemeinen Ausführungsfrequenz von Codestelle vielleicht unmöglich zu analysieren.

Oft geht es aber, z.B. nach einer der folgenden Regeln

Code	Laufzeit T
<code>x = 0, a[i]</code> <code>x + y, x * y, x++</code> etc.	1 (falls Teil des abstrakte Kostenmaßes, sonst 0)
1 <code>for (int j = a; j <= b; ++j) {</code> 2 <code> B(j) // weiterer Code</code> 3 <code>}</code>	$\sum_{j=a}^b T_B(j)$ für $T_B(j)$ Laufzeit des Schleifenrumpfs mit Wert j
1 <code>if (...) { A } else { B }</code>	$\leq \max\{T_A, T_B\}$

Von Code zu Analyse

Achtung: Im Allgemeinen Ausführungsfrequenz von Codestelle vielleicht unmöglich zu analysieren.

Oft geht es aber, z.B. nach einer der folgenden Regeln

Code	Laufzeit T
<code>x = 0, a[i]</code> <code>x + y, x * y, x++</code> etc.	1 (falls Teil des abstrakte Kostenmaßes, sonst 0)
<pre>1 for (int j = a; j <= b; ++j) { 2 B(j) // weiterer Code 3 }</pre>	$\sum_{j=a}^b T_B(j)$ für $T_B(j)$ Laufzeit des Schleifenrumpfs mit Wert j
<pre>1 if (...) { A } else { B }</pre>	$\leq \max\{T_A, T_B\}$
<pre>1 while (n > c) { 2 A(n) // weiterer Code 3 n = n/2; 4 B(n) // weiterer Code 5 }</pre>	

Von Code zu Analyse

Achtung: Im Allgemeinen Ausführungsfrequenz von Codestelle vielleicht unmöglich zu analysieren.

Oft geht es aber, z.B. nach einer der folgenden Regeln

Code	Laufzeit T
<pre>x = 0, a[i] x + y, x * y, x++ etc.</pre>	1 (falls Teil des abstrakte Kostenmaßes, sonst 0)
<pre>1 for (int j = a; j <= b; ++j) { 2 B(j) // weiterer Code 3 }</pre>	$\sum_{j=a}^b T_B(j)$ für $T_B(j)$ Laufzeit des Schleifenrumpfs mit Wert j
<pre>1 if (...) { A } else { B }</pre>	$\leq \max\{T_A, T_B\}$
<pre>1 while (n > c) { 2 A(n) // weiterer Code 3 n = n/2; 4 B(n) // weiterer Code 5 }</pre>	
<pre>1 void m(n) { 2 if (n <= c) return; 3 A(n) // weiterer Code 4 m(n/2); // Rekursion 5 B(n/2) // weiterer Code 6 }</pre>	

Von Code zu Analyse

Achtung: Im Allgemeinen Ausführungsfrequenz von Codestelle vielleicht unmöglich zu analysieren.

Oft geht es aber, z.B. nach einer der folgenden Regeln

Code	Laufzeit T
<pre>x = 0, a[i] x + y, x * y, x++ etc.</pre>	1 (falls Teil des abstrakte Kostenmaßes, sonst 0)
<pre>1 for (int j = a; j <= b; ++j) { 2 B(j) // weiterer Code 3 }</pre>	$\sum_{j=a}^b T_B(j) \quad \text{für } T_B(j) \text{ Laufzeit des Schleifenrumpfs mit Wert } j$
<pre>1 if (...) { A } else { B }</pre>	$\leq \max\{T_A, T_B\}$
<pre>1 while (n > c) { 2 A(n) // weiterer Code 3 n = n/2; 4 B(n) // weiterer Code 5 }</pre>	Für allgemeine While-Schleifen und rekursive Methoden
	Rekursionsgleichung
<pre>1 void m(n) { 2 if (n <= c) return; 3 A(n) // weiterer Code 4 m(n/2); // Rekursion 5 B(n/2) // weiterer Code 6 }</pre>	$T(n) = \underbrace{T(n/2)} + T_A(n) + T_B(n/2) \quad (n > c)$ $T(n) = 1 \quad (n \leq c)$ ↪ Master-Theorem

Example: 2-SUM

Q. Approximately how many **array accesses** as a function of input size N ?

```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```

Annotations: A red bracket on the left of the inner loop is labeled "inner loop". A vertical line separates the inner loop body from the iteration count. A handwritten 2 is next to the inner loop body, and $T_B(i,j) = 2$ is written to its right.

$$T_C(i) = \sum_{j=i+1}^{N-1} T_B(i,j)$$

"2"

$$0 + 1 + 2 + \dots + (N-1) = \frac{1}{2}N(N-1) = \binom{N}{2}$$

A. $\sim N^2$ array accesses.

$$\sum_{i=0}^{N-1} T_C(i)$$

Bottom line. Use cost model and tilde notation to simplify counts.

Example: 3-SUM

Q. Approximately how many **array accesses** as a function of input size N ?

```
int count = 0;
for (int i = 0; i < N; i++)
  for (int j = i+1; j < N; j++)
    for (int k = j+1; k < N; k++)
      if (a[i] + a[j] + a[k] == 0) ← "inner loop"
        count++;
```

A. $\sim \frac{1}{2} N^3$ array accesses.

$$\binom{N}{3} = \frac{N(N-1)(N-2)}{3!}$$
$$\sim \frac{1}{6} N^3$$

Bottom line. Use **cost model** and **tilde notation** to simplify counts.